

Smart Real Estate Assistant

Έξυπνος Βοηθός Αγοράς Ακινήτων

Τεχνική Τεκμηρίωση & Τελική Εργασία

AI for Developers — Athens University of Economics and Business

1. Τίτλος, Περιγραφή και Σκοπός

Το Smart Real Estate Assistant είναι μια εφαρμογή Generative AI που λειτουργεί ως ψηφιακός σύμβουλος για την ελληνική αγορά ακινήτων. Επιτρέπει στους χρήστες να αποκτούν τόσο αναλύσεις αγοράς όσο και οικονομικούς υπολογισμούς μέσω συνομιλίας σε φυσική γλώσσα (ελληνικά ή αγγλικά).

Το σύστημα δεν είναι ένα απλό chatbot — είναι ένας Agentic AI που αποφασίζει μόνος του ποια εργαλεία θα χρησιμοποιήσει για κάθε ερώτηση. Μπορεί να αναζητήσει πληροφορίες στη βάση γνώσης (RAG) ή να εκτελέσει υπολογισμούς δανείου, ή και τα δύο στην ίδια απάντηση.

Κύριες Λειτουργίες

- Ερωτήσεις αγοράς ακινήτων: τιμές, τάσεις, γειτονιές, νομικό πλαίσιο
- Υπολογισμός στεγαστικών δανείων: μηνιαία δόση, συνολικό κόστος, πίνακας αποπληρωμής
- Συνδυαστικές απαντήσεις: αγορά + δάνειο σε ένα ερώτημα
- Authenticated multi-user: κάθε χρήστης έχει ξεχωριστό ιστορικό συνομιλιών
- Session memory: ο agent θυμάται το πλαίσιο της συνομιλίας για follow-up ερωτήσεις

2. Βασικό Σενάριο Χρήσης

Ένας υποψήφιος αγοραστής ακινήτου θέλει να εξερευνήσει την αγορά στην Αθήνα και να κατανοήσει αν μπορεί να αντέξει οικονομικά ένα ακίνητο. Αντί να ψάχνει σε πολλαπλές πηγές, κάνει ερωτήσεις στον assistant:

- "Ποιες είναι οι τιμές στα νότια προάστια;" → ο agent αναζητά στη βάση γνώσης
- "Αν αγοράσω στα 300.000€ με επιτόκιο 4%, τι δόση θα πληρώνω;" → υπολογισμός
- "Ποιο είναι το minimum για Golden Visa;" → RAG από αρχεία κανονισμών

Ο χρήστης εγγράφεται, συνδέεται, δημιουργεί sessions ("Ερευνα Γλυφάδα", "Σύγκριση Παγκράτι") και επιστρέφει αργότερα για να δει το ιστορικό του.

Λειτουργικές Απαιτήσεις

- Εγγραφή/σύνδεση χρήστη με ασφαλή authentication
- Δυνατότητα εισαγωγής εγγράφων αγοράς στη βάση γνώσης (admin)
- Φυσική γλώσσα — χωρίς δομημένες φόρμες
- Αποθήκευση ιστορικού ανά session, με δυνατότητα μετονομασίας και διαγραφής
- Παρουσίαση αποτελεσμάτων σε κατανοητή μορφή με αριθμούς και ευρώ

3. Τεχνολογίες και Αιτιολόγηση Επιλογών

Στρώμα	Τεχνολογία	Αιτιολόγηση
Backend API	FastAPI (Python 3.12)	Async-first, αυτόματο Swagger/OpenAPI, Pydantic validation, ideal για AI backends
AI Agent	LangGraph + LangChain	Stateful graph για agentic loops (agent → tools → agent), ίδιο stack με το bootcamp
Κύριο LLM	Claude Sonnet 4.6 (Anthropic)	Υψηλή ποιότητα function calling, ελληνική γλώσσα, tool use
Fallback LLM	GPT-4o Mini (OpenAI)	Αυτόματο fallback αν το primary API αποτύχει
Embeddings	text-embedding-3-small (OpenAI)	Υψηλή ποιότητα, χαμηλό κόστος, ελληνική υποστήριξη
Vector Store	ChromaDB (persistent)	In-process, zero infrastructure, cosine similarity, εύκολο category filtering
ORM / DB	SQLModel + PostgreSQL	Pydantic-native ORM, τύπους schemas ως DB models
Authentication	JWT + Argon2 (pwdlib)	Industry standard, timing-attack safe, password rehashing
UI	Gradio 5 (mounted στο FastAPI)	Γρήγορο Python UI, mounted στο ίδιο process, session state per browser tab

4. Αρχιτεκτονική και Ροή Δεδομένων

4.1 Δομή Project

```

app/
  main.py          # FastAPI app factory + Gradio mount
  core/            # config.py, db.py, security.py
  models/          # SQLModel tables: User, SearchSession, SearchHistory
  schemas/         # Pydantic DTOs: request/response schemas
  api/
    dependencies.py # SessionDep, CurrentUser, superuser gate
    routers/        # auth, users, chat, sessions, knowledge, retrieval, health
  agent/graph.py   # LangGraph StateGraph
  rag/             # vector_store.py, pipeline.py
  tools/           # mortgage_calculator.py, retriever_tool.py
  ui/              # gradio_app.py, api_client.py
  tests/           # 63 pytest tests

```

4.2 Ροή Δεδομένων — Chat Turn

1. Χρήστης → POST /chat {message, session_id} (με JWT Bearer token)
2. chat.py router → φορτώνει SearchSession.memory (JSON cache από PostgreSQL)
3. Deserializes memory → LangChain HumanMessage/AIMessage objects

- LangGraph invoke: agent node → αποφασίζει εργαλείο → tools node → agent node (loop)
- Εργαλεία: search_knowledge_base (ChromaDB cosine search) ή calculate_mortgage
- Τελικό AIMessage → εξάγεται reply_text και tools_used metadata
- Persist: ενημερώνεται memory (capped JSON) + νέο SearchHistory row
- Επιστροφή ChatResponse στον client

4.3 Dual Persistence Architecture

Για την αποθήκευση ιστορικού χρησιμοποιούνται δύο συμπληρωματικοί μηχανισμοί:

- Fast path:** SearchSession.memory (JSON text field): γρήγορο cache των τελευταίων MAX_MEMORY_TURNS turns για τον agent. Single-row read, no JOINS.
- Audit trail:** SearchHistory (PostgreSQL table): durable, queryable audit trail με ένα row ανά turn. Timestamps, filtering, admin analytics.

5. FastAPI Endpoints και Services

5.1 Authentication Endpoints

Method	Endpoint	Περιγραφή	Auth
POST	/login/access-token	OAuth2 login, επιστρέφει JWT token	—
POST	/users/signup	Εγγραφή νέου χρήστη	—
POST	/login/test-token	Επαλήθευση token	Bearer
POST	/password-recovery/{email}	Αποστολή email επαναφοράς	—
POST	/reset-password/	Επαναφορά password με token	—

5.2 Chat & Session Endpoints

Method	Endpoint	Περιγραφή
POST	/chat	Αποστολή μηνύματος στον AI agent (session-based)
POST	/sessions/	Δημιουργία νέας συνομιλίας
GET	/sessions/	Λίστα συνομιλιών τρέχοντος χρήστη
GET	/sessions/{id}	Λήψη συγκεκριμένης συνομιλίας
PATCH	/sessions/{id}	Μετονομασία συνομιλίας
DELETE	/sessions/{id}	Διαγραφή συνομιλίας (cascade)

Method	Endpoint	Περιγραφή
GET	/sessions/{id}/history	Turn-by-turn ιστορικό (SearchHistory rows)

5.3 Knowledge Base Endpoints (Admin only)

Method	Endpoint	Περιγραφή
POST	/knowledge/ingest	Εισαγωγή εγγράφου (chunk+embed+store)
POST	/knowledge/ingest/batch	Μαζική εισαγωγή εγγράφων
GET	/knowledge/stats	Στατιστικά collection (categories, sources)
DELETE	/knowledge/reset	Εκκαθάριση ChromaDB collection
POST	/retrieval/query	Σημασιολογική αναζήτηση (debug)
POST	/retrieval/context	LLM-ready context string (debug)
GET	/health	Κατάσταση συστήματος + RAG metrics

6. User Interface — Gradio

Το UI κατασκευάστηκε με Gradio 5 Blocks και είναι mounted απευθείας στο FastAPI app στο path /ui. Δεν χρειάζεται ξεχωριστή εκτέλεση — λειτουργεί ως μέρος του ίδιου Unicorn process.

6.1 Layout

- Logged-out view: Login tab (email + password) και Signup tab (email + password + full name)
- Logged-in view: welcome bar με logout button, sidebar με session management, και κεντρικό chat panel

6.2 Session Sidebar

- "✚ New chat" button: δημιουργεί νέο SearchSession
- Radio list: επιλογή υπάρχουσας συνομιλίας → φορτώνει ιστορικό
- Rename field + button: μετονομασία session
- Delete button: διαγραφή session + cascade σε SearchHistory

6.3 Chat Panel

- Chatbot component (type='messages'): εμφανίζει user/assistant turns
- Message textbox: submit με Enter ή Send button
- Auto-scroll, handles empty messages, API error display

6.4 State Management

Κάθε browser session διατηρεί ξεχωριστό state μέσω `gr.State`: `token`, `user info`, `active session_id`, `sessions list`. Ποτέ δεν χρησιμοποιούνται `module-level globals` — `concurrent users` δεν αλληλεπιδρούν.

6.5 API Client (`app/ui/api_client.py`)

Το Gradio UI επικοινωνεί με το backend μέσω `httpx` HTTP calls στο `http://127.0.0.1:{APP_PORT}`. Αυτή η αρχιτεκτονική διατηρεί το UI ως κανονικό API consumer — ίδιο με οποιαδήποτε άλλη εφαρμογή που θα χρησιμοποιούσε το API.

7. GenAI Τεχνικές

7.1 Prompt Engineering — System Prompt

Ο agent λειτουργεί με `role-based system prompt` που ορίζει:

- Ρόλο: "Smart Real Estate Assistant ειδικευμένος στην ελληνική αγορά"
- Πότε να χρησιμοποιεί κάθε εργαλείο (αγορά vs. δάνειο)
- Μορφοποίηση αποτελεσμάτων (€ symbol, πίνακες αποπληρωμής)
- Πολυγλωσσικότητα: απαντά στη γλώσσα που γράφει ο χρήστης

7.2 RAG — Retrieval Augmented Generation

Αρχιτεκτονική RAG pipeline:

- Ingest: κείμενα χωρίζονται σε `chunks` ~200 χαρακτήρων με 40-χαρακτήρες `overlap` (sentence-boundary aware)
- Embedding: `OpenAI text-embedding-3-small` → 1536-dimensional vectors
- Storage: `ChromaDB` persistent collection με `cosine similarity metric`
- Retrieval: `top-K=5` chunks πάνω από `similarity threshold 0.3`
- Category filtering: `market_data`, `neighborhood`, `legal`, `general`
- Context injection: chunks μορφοποιούνται με `source/score` πληροφορίες πριν το LLM prompt

7.3 AI Agents — LangGraph StateGraph

Ο agent υλοποιείται ως `LangGraph StateGraph` με το παρακάτω topology:

```
START → agent_node → should_continue?  
                                ↓ tool_calls present → tools_node → agent_node (loop)  
                                ↓ no tool_calls      → END
```

- `agent_node`: καλεί `Claude Sonnet 4.6` με πλήρες `message history`. Αν αποτύχει → `GPT-4o Mini fallback`
- `tools_node`: εκτελεί τα εργαλεία που επιλέχθηκαν από τον agent (`ToolNode`)
- `should_continue`: edge function — αν υπάρχουν `tool_calls` → `tools`, αλλιώς `END`
- `AgentState`: `TypedDict` με `messages list` και `add_messages reducer`

7.4 Tool Calling — Function Calling

Δύο tools ορίζονται με @tool decorator και bind_tools() στο LLM:

search_knowledge_base(query, category)

- Εκτελεί semantic search στο ChromaDB
- Επιστρέφει formatted context string με sources και scores
- Ο agent το καλεί για ερωτήσεις αγοράς ακινήτων

calculate_mortgage(loan_amount, annual_interest_rate, loan_term_years, down_payment)

- Standard annuity formula: $M = P * [r(1+r)^n] / [(1+r)^n - 1]$
- Επιστρέφει μηνιαία δόση, συνολικό κόστος, τόκους, amortisation snapshots ανά 5 χρόνια
- Ο agent το καλεί για υπολογισμούς δανείου

7.5 Conversation Memory

Η μνήμη συνομιλίας διατηρείται server-side (όχι client-side) σε δύο επίπεδα:

- SearchSession.memory: JSON-serialized list των ChatMessage objects, capped στα MAX_MEMORY_TURNS turns. Φορτώνεται σε single DB query χωρίς JOIN για fast agent warm-up.
- SearchHistory: ένα PostgreSQL row ανά turn με query, result, session_id, owner_id, timestamp. Χρησιμοποιείται για full history retrieval και admin analytics.

8. Οδηγίες Εγκατάστασης και Εκτέλεσης

8.1 Προαπαιτούμενα

- Python 3.12+
- PostgreSQL 14+
- Anthropic API key (Claude Sonnet 4.6)
- OpenAI API key (embeddings + GPT-4o Mini fallback)

8.2 Εγκατάσταση

```
git clone https://github.com/<username>/smart-real-estate-assistant.git
cd smart-real-estate-assistant
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # συμπλήρωσε API keys και DB credentials
```

8.3 Εκτέλεση

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

- API + Swagger: <http://localhost:8000/docs>
- Gradio UI: <http://localhost:8000/ui>
- Ο πρώτος superuser (FIRST_SUPERUSER) δημιουργείται αυτόματα κατά την εκκίνηση

8.4 Εισαγωγή Δεδομένων στη Βάση Γνώσης

Μέσω Swagger UI (/docs) ή curl, ως superuser:

```
curl -X POST http://localhost:8000/knowledge/ingest \  
-H "Authorization: Bearer <token>" \  
-H "Content-Type: application/json" \  
-d '{"content": "Τιμές Κολωνάκι 2026...", "source": "report.txt", "category":  
"market_data"}'
```

8.5 Tests

```
pytest tests/ -v
```

63 tests σε 6 αρχεία — τρέχουν χωρίς PostgreSQL (SQLite in-memory fixtures).

9. Παραδείγματα Χρήσης

9.1 Ερώτηση αγοράς ακινήτων

Ερώτηση: "Ποιες είναι οι τιμές ενοικίασης στο Παγκράτι;"

Agent action: καλεί `search_knowledge_base(query="τιμές ενοικίασης Παγκράτι")`

Απάντηση:

Σύμφωνα με την Αναφορά Αγοράς 2026, οι τιμές ενοικίασης στο Παγκράτι κυμαίνονται μεταξύ 10-12€/τ.μ. για ανακαινισμένα διαμερίσματα. Υπάρχει υψηλή ζήτηση για 50-60 τ.μ. λόγω φοιτητών και βραχυπρόθεσμων μισθώσεων.

9.2 Υπολογισμός στεγαστικού δανείου

Ερώτηση: "Δάνειο 200.000€, επιτόκιο 3.5%, 20 χρόνια — ποιά η δόση;"

Agent action: καλεί `calculate_mortgage(loan_amount=200000, annual_interest_rate=3.5, loan_term_years=20)`

Απάντηση:

Μηνιαία δόση: 1.159,92 €
Συνολικό κόστος: 278.380,80 €
Συνολικοί τόκοι: 78.380,80 €
Έτος 5: υπόλοιπο 160.847€ | Έτος 10: 114.472€ | Έτος 20: 0€

9.3 Συνδυαστική ερώτηση

Ερώτηση: "Τιμές νοτίων προαστίων και δόση για 350.000€ με προκαταβολή 70.000€, 4% για 25 χρόνια;"

Agent action: καλεί ΚΑΙ τα δύο εργαλεία — search_knowledge_base + calculate_mortgage

Απάντηση:

Νότια προάστια (Γλυφάδα, Βούλα): τιμές πώλησης >4.500€/τ.μ.

Δάνειο 280.000€ (350.000 - 70.000 προκαταβολή):

Μηνιαία δόση: 1.476,96€ | Συνολικοί τόκοι: 163.088€

10. Περιορισμοί

- Η ποιότητα RAG εξαρτάται από τα έγγραφα που εισάγει ο admin — χωρίς ingested data, ο agent απαντά από γενική γνώση
- Δεν υποστηρίζεται PDF ingest από το UI — τα έγγραφα εισάγονται ως plain text μέσω API
- Η Gradio UI δεν έχει responsive mobile design
- Η ταχύτητα απόκρισης εξαρτάται από LLM latency (~3-8 δευτερόλεπτα)
- Το ChromaDB τρέχει in-process — για production απαιτείται Chroma server ή Qdrant
- Δεν υπάρχει streaming — η απάντηση εμφανίζεται ολόκληρη όταν τελειώσει ο agent

11. Μελλοντικές Επεκτάσεις

- PDF upload από το UI: ο ADMIN ανεβάζει αρχεία αγοράς και εισάγονται αυτόματα
- Streaming responses: token-by-token εμφάνιση απάντησης για καλύτερο UX
- Tavily web search tool: real-time αναζήτηση για τρέχουσες τιμές αγοράς
- Admin dashboard: γραφική διαχείριση βάσης γνώσης, analytics χρηστών
- React/Vue frontend: αντικατάσταση Gradio με πλήρες SPA για production
- Docker + docker-compose: containerized deployment με PostgreSQL + ChromaDB server
- Alembic migrations: πλήρες migration history για production DB management